

Sentinel

Every answer comes with its evidence.

A document checker that is confidently wrong is worse than no checker at all - because nobody goes back and looks.

DuoCode · Tang Chye Fong · Lim Yee Teng · Asia Pacific University

duocode-sentinel.vercel.app · github.com/TCF1209/duocode-sentinel

Averis x Monash Hackathon 2026 · Final Round

The problem - and the fourth case

01 Classify

Tell message types apart: comparison requests, new SI requests, invoice queries, general mail, spam.

02 Extract

For comparison requests, read the SI and BL attachments and pull the matching shipment fields.

03 Compare

Surface any mismatched fields, showing the SI and BL values side by side.

04 Ask for help

When it can't finish on its own, escalate to a person with full context instead of guessing or failing silently.

The organisers' own four capabilities. The SI is the reference; the goal is to catch incorrect details before the draft is finalised.

Finding the right emails takes time

A document request that is overlooked never reaches the checking step.

Manual comparison is easy to get wrong

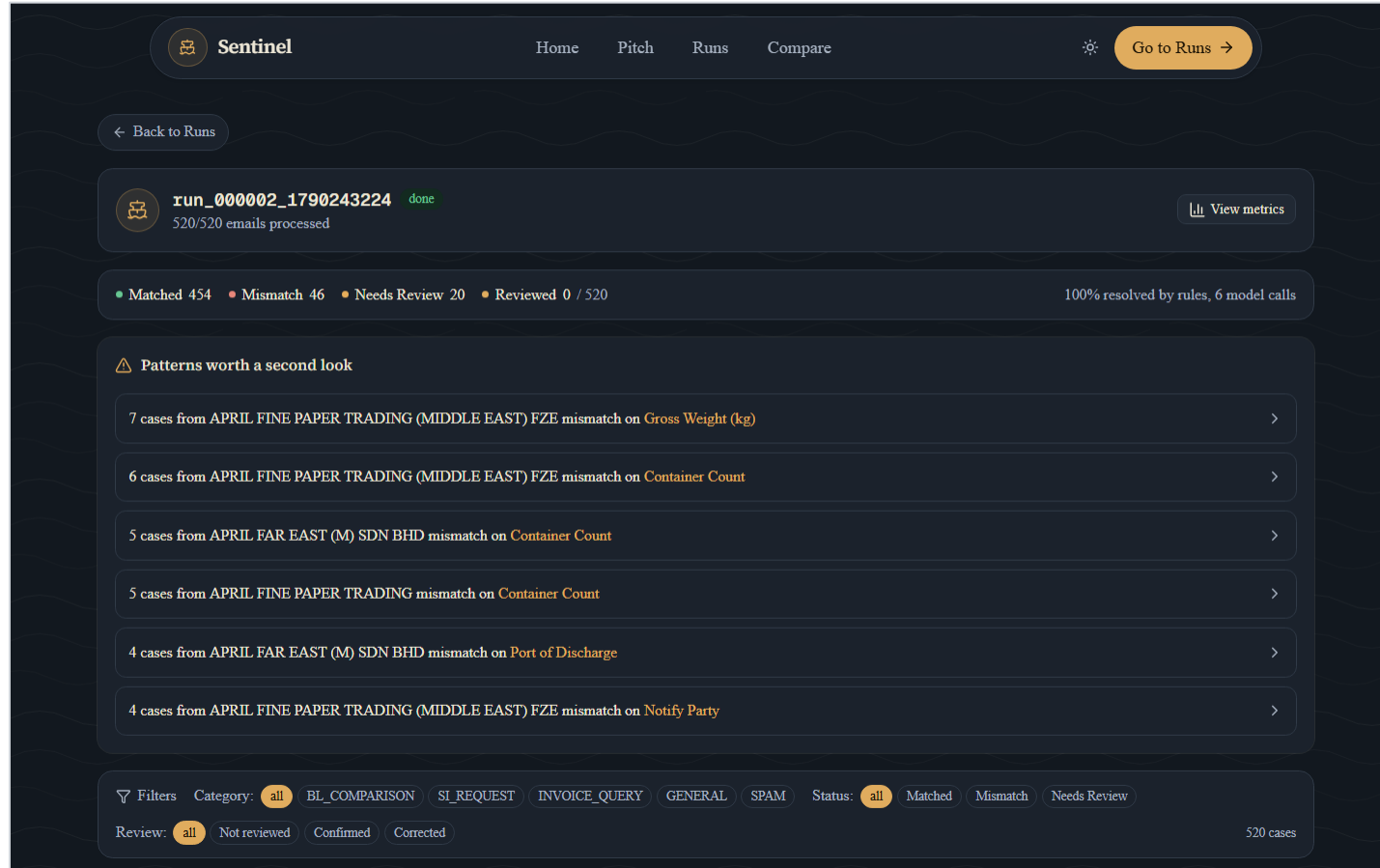
Names, ports, quantities and weight across two documents; a missed discrepancy is a correction, a delay, rework.

The same fact looks different

One document says 'Port of Loading', the other 'Load Port' - the system has to know they are the same field.

The fourth case: sometimes the check cannot be done at all - an unreadable scan, a blank field, the wrong document. That has to reach a person with the reason attached, not be guessed at. Sentinel does all four.

What it does, end to end



520

emails, classified and routed

46

mismatches, with the fields that differ

20

sent to a person - with the reason and what to do

12.7 s

for the whole inbox, on a free-tier container

duocode-sentinel.vercel.app · sdcc-sentinel-api.onrender.com - deployed from the repository by the committed render.yaml and vercel.json.

How it decides



Six stages, one direction. The gate runs after the comparison: a value we cannot find again in the document it was read from is never reported as a discrepancy - it becomes a question for a person, with both readings attached.

Labels by meaning, values exactly

Never a similarity score. A threshold loose enough to forgive a scan artefact also merges two real companies - and this data has them.

Rules first, model second

Every stage tries a deterministic answer before a model, and records which one answered (decided_by). The model is asked only where the rules admit they cannot read.

One stateless pipeline library

No web framework or database in backend/sdoc. The CLI, the API and the tests run the same code, so the thing scored is the thing demonstrated.

Scaling, as built: state sits behind one class in one file (store.py) - Postgres is a one-file change · one worker on purpose · throughput is more copies of a stateless library · cost does not scale with volume on this inbox · per-desk rules slot into the existing stage boundaries.

Where the AI is - and why it is aimed

C

Classify - an email the rules cannot separate

asked only when the rule score is ambiguous; a closed five-way answer, never free text

R

Read - a field label the table has never seen

asked only about fields no rule resolved; every value must be found again in the document before it is adopted

S

See - a scanned page with no text layer

transcribed for the reviewer, never fed into a comparison - the case stays in review

168 → 2

cases forced to a human on wording we invented - rules alone vs rules + model, 188 documents

False discrepancies: 0 both ways. Recall bought by guessing would have shown up there; it didn't.

gpt-5-mini, structured output. One rule for all three: nothing the model returns is adopted until it is found again in the document.

\$0.0013 per document at the published rates · cached · \$2 ceiling per run. On this inbox: 0 classifier calls, 0 extractor calls, 6 scans read out for the reviewer - every decision is a rule's, and that is measured, not assumed.

How we know it holds

Not memorised

1.0000 on four datasets, three from seeds we never developed against - 225 planted defects caught with the exact field set, 80/80 escalations correct.

Attacked ourselves

16 kinds of damage, 3,008 perturbed documents, 20,496 field reads, no answer key. Thirteen modes at zero movement; silent wrong values 982 -> 0.

A real carrier's form

CMA CGM's public SI template, from outside the generator: four fields held, one bug found and fixed the same day.

Engineering

596 tests, 0 failing; CI on every push; a container that runs as a non-root user with no secret baked in; every value carries its evidence.

What we haven't fixed

email_145: a wrapped party name cut short to exactly what the other document says leaves the repair nothing to repair, and one real mismatch in 3,008 perturbed documents is reported as a match.

The obvious guard would flag 92% of genuine party fields - worse than the gap - so it stays open and written down (ADVERSARIAL.md 5.2).

A defect we hide is worse than one we miss.

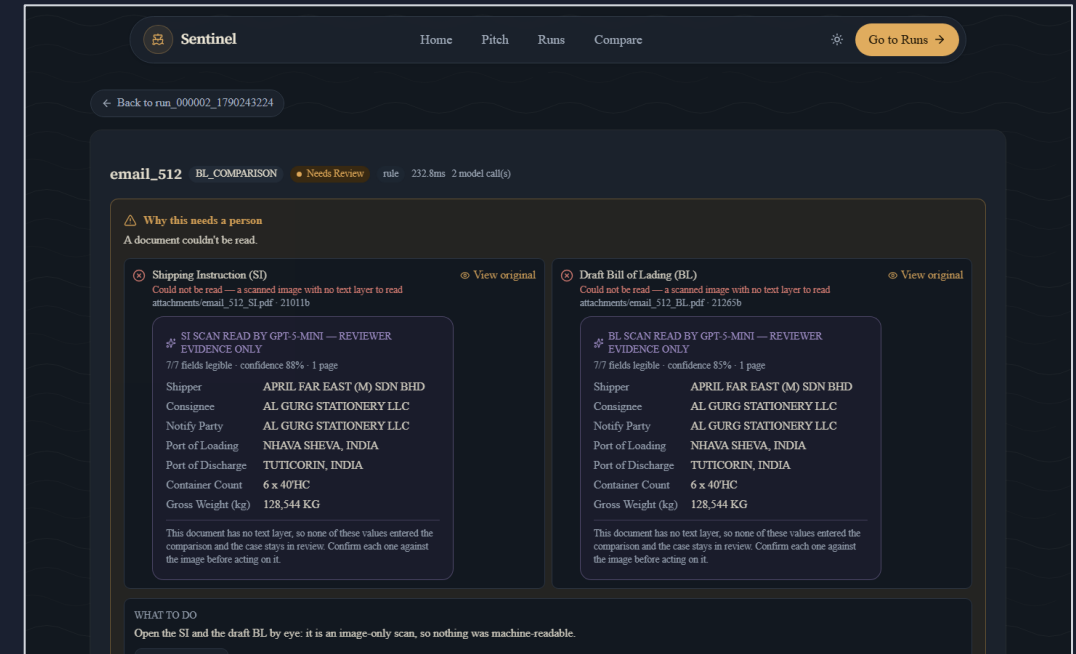
A perfect score on the dataset you were handed proves you didn't memorise it. It doesn't prove the reader works - so we went looking for the failures ourselves.

Live demo

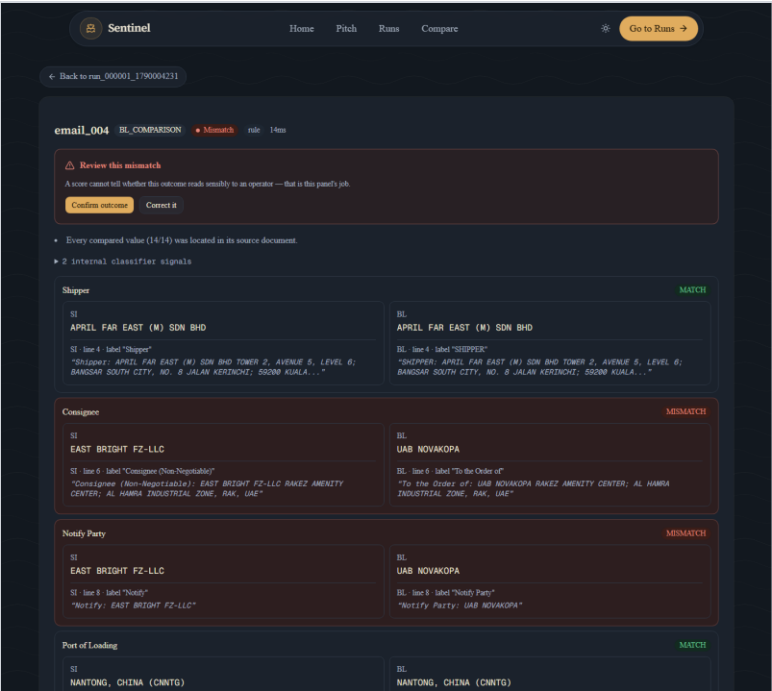
90 seconds, on the deployed URLs

- 1 Start a run with the model tier on - the organisers' full inbox, 520 emails
- 2 Patterns worth a second look - one shipper, one field, six times
- 3 A mismatch case - under every value, the line it was read from
- 4 A scanned case - the page read out for the reviewer; the case still in review, decided per field
- 5 Compare, unfamiliar labels: model off, then on - 'model answered'

Fallback if the venue network fails: the same inbox on the laptop, 1.5 s, no network - only the model beat changes.

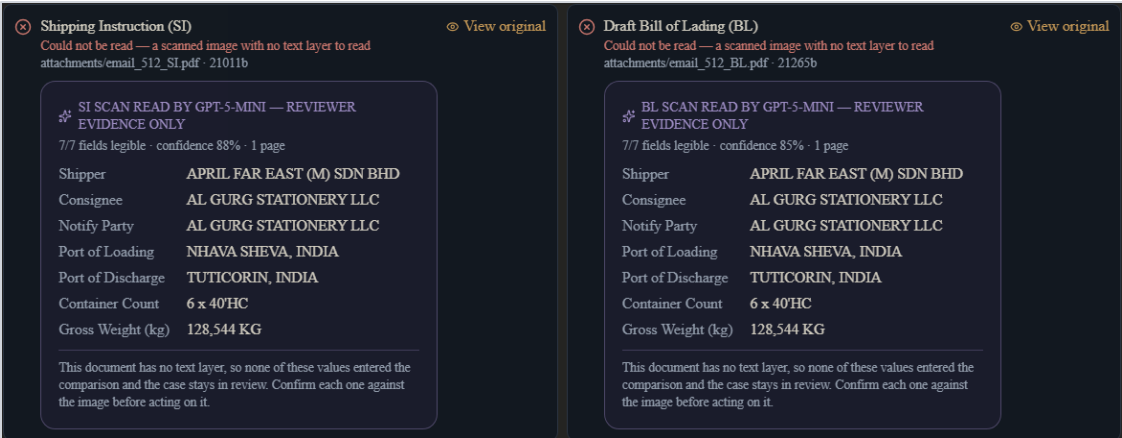


What a reviewer gets



A mismatch: the two fields that differ, and under every value the document, the line and the label it was read from.

- Correct a single field, with how often this shipper was wrong on it beside the box
- Patterns worth a second look: the same shipper's repeated mistakes, grouped
- A reply draft built from the corrected outcome, not the stale one
- Every escalation carries the reason and what to do about it



A scan: the model's reading as reviewer evidence, marked as such - nothing entered the comparison.

Impact, and what comes next

11 hours → 13 seconds

520 emails, 124 document pairs. Eleven hours is a conservative estimate (20 s an email, 4 min a pair), not a measurement; thirteen seconds is measured. Every decision on that inbox was a rule's.

First deployment: one desk, not a region

The inbox already carries four desk codes (AFEMY 35, AIE 30, AFRT 29, AFPTME 22 emails). Sentinel sits beside the desk's existing check until the measures below have held for a full cycle of its carriers.

Measure	Today (graded inbox)	The pilot watches
Escalation rate	20 of 220 requests (9.1%), all correct	precision at 1.0 as unfamiliar templates grow
False alarms	0 of 46 defects; 0 across 16 perturbation modes	the weekly number
Reviewer minutes per escalation	not measured yet	the pilot's first new measurement

Cheap because the model is aimed, not because it is absent.

Next, in order

- 1

Per-desk rules
a label table and an escalation policy per desk, selected by the code the inbox already carries
- 2

Corrections feed the label table
every confirmed correction is a labelled example of wording we could not read - the one place this system should learn
- 3

The database
one file; the demo does not need it yet, a pilot will

Every answer comes with its evidence.

Sentinel, by DuoCode

github.com/TCF1209/duocode-sentinel · duocode-sentinel.vercel.app